
포트홀 탐지 yolov8n 모델 개선

Improvement of the YOLOv8n Model for Pothole Detection

이 정 인, 정 진 용, 지 시 현, 나 덤

-목차-

I. 서론	1
II. 연구 방법	3
2.1 데이터셋 구성	3
2.2 데이터 전처리	3
2.3 모델 학습	5
2.3.1 최적화 기법 설정	5
2.3.2 손실 함수 수정	6
2.3.3 백본 구조 수정	8
2.3.4 활성화 함수 수정	9
III. 실험 결과	10
3.1 실험 환경	10
3.2 학습 결과 분석	11
3.3 세 후보 모델 간 비교	11
3.4 원본 YOLOv8n 모델과의 비교	12
IV. 결론	12

I. 서론

본 연구의 목적은 주행 중 포트홀을 탐지할 수 있는 경량화된 모델을 개발하는 것이다. 차량 및 이륜 자동차에 장착 가능하며, 실시간 탐지가 가능한 모델을 설계하고자 한다.

II. 연구 방법

2.1 데이터셋 구성 (Dataset Construction)

2.1.1 데이터셋 선정 기준

포트홀 탐지를 위해 본 연구에서는 자동차 및 이륜자동차의 전방 시점에서 촬영된, 실제 포트홀이 포함된 데이터를 선정하였다. 드론이나 항공 촬영 이미지 등은 연구 목적과 맞지 않아 제외하였다. 데이터는 주간과 야간, 다양한 도로 환경에서 촬영되었으며, 다양한 크기와 형태의 포트홀이 포함되어 있다. 시야가 확보되지 않은 흐린 이미지나 심하게 가려진 이미지는 제외하였다.

2.1.2 데이터셋 소개 및 특성

본 연구에서는 AI HUB에서 제공하는 ‘지자체 도로 정비 AI 학습용 데이터’와 ‘이륜자동차 안전 위험 시설물 데이터’를 활용하였다. 확보한 데이터는 포트홀 Detection용 이미지로 구성되어 있으며, 라벨링이 불완전하거나 부정확한 데이터는 연구자가 직접 수정하였다. 이후 학습의 효율성을 높이기 위해 데이터셋을 TRAIN: 528장, VAL: 132장으로 분할하였으며, 이는 8:2 비율에 해당한다. 또한, 모델이 포트홀과 포트홀이 아닌 노면 환경(노면 오염, 맨홀 등)을 구분할 수 있도록 배경 이미지를 추가하였으며, TRAIN: 172장, VAL: 43장으로 구성하였다.

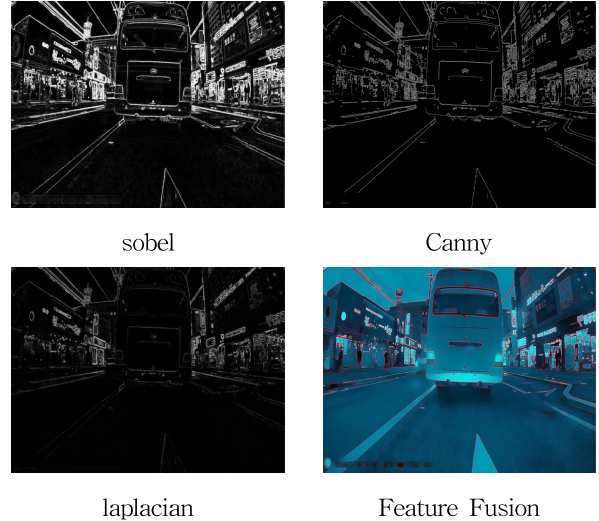
2.2 데이터 전처리

2.2.1 이미지 전처리 과정

아래의 4가지 전처리 방법(Sobel, Canny, Laplacian, Feature Fusion)은 기존 포트홀 탐지 연구 논문들에서 자주 사용된 기법들을 참고하여 본 연구의 데이터셋에도 적용해 본 것이다. 이때 Feature Fusion 전처리는 Sobel, Supersixel, Grayscale로 각각 추출한 경계·형태·질감 정보를 R·G·B 채널에 배치해 하나의 3채널 이미지로 합성하는 방식이다. 그러나 본

연구에서 사용한 데이터셋은 전체적으로 해상도가 낮고 노이즈가 많아, 이러한 전처리들을 적용했을 때 포트홀보다 도로 차선이나 주변 구조물이 더 강하게 드러나거나 포트홀 윤곽이 희미해져 선이 제대로 이어지지 않는 문제가 발생하였다. 그림에서 보이는 것처럼 포트홀 윤곽이 충분히 강조되지 않았기 때문에, 본 연구에서는 이 전처리 방법들을 사용하지 않도록 하였다.

Figure 1 _ 다양한 전처리 기법 적용



이러한 한계를 보완하기 위해 본 연구에서는 데이터셋에 포함된 이미지에 CLAHE(Contrast Limited Adaptive Histogram Equalization)를 적용하는 전처리 방식을 사용하였다. 구체적으로, 원본 BGR 이미지를 YCrCb 색상 공간으로 변환한 뒤 밝기 성분인 Y 채널만 분리하여 CLAHE를 적용하고, 처리된 Y 채널을 다시 원본의 Cr, Cb 채널과 합쳐 BGR 이미지로 되돌리는 방식이다. 이때 CLAHE의 clipLimit은 0.2, tileGridSize는 (8, 8)로 설정하여 대비를 과도하게 키우지 않으면서도 어두운 영역의 명암 차이를 부드럽게 확장하도록 하였다. 이러한 과정을 거친 뒤, 아래 그림에서 보이듯 전체 밝기가 보다 고르게 분포하고 어두운 영역의 텍스처가 선명해져, 원본 이미지에 비해 포트홀 형태가 더 잘 드러나는 것을 확인할 수 있었다.

Figure 2 _ 원본과 CLAHE 전처리 적용 결과 비교



각 전처리 방법이 모델 성능에 미치는 영향을 정량적으로 확인하기 위해 Original, CLAHE, Sobel, Canny, Laplacian, Feature Fusion 각각을 적용하여 동일한 조건에서 학습을 진행하고 성능을 비교하였다. Table 1에서 CLAHE가 Precision(0.871), mAP50(0.850), mAP50-95(0.523) 등 대부분의 지표에서 가장 우수한 성능을 보였으며, Original 역시 그다음으로 안정적인 성능을 보였다. 반면 Sobel, Canny, Laplacian, Feature Fusion을 적용한 경우에는 시각적 결과에서와 마찬가지로 포트홀보다 주변 구조가 더 많이 검출되는 특성이 그대로 반영되어 Recall과 mAP 지표가 전반적으로 낮게 나타났다. 이러한 결과를 통해, 화질이 낮고 노이즈가 많은 본 연구의 데이터셋에서는 CLAHE 전처리가 포트홀 탐지 성능을 향상시키는 데 가장 효과적인 방법임을 확인하였다.

Table 1 기본 모델에 전처리 적용한 결과

PreProcessing Method	best Epoch	Precision	Recall	map 50	map 50-95
Original	55	0.797	0.732	0.807	0.465
clahe	78	0.871	0.739	0.850	0.523
sobel	30	0.563	0.369	0.393	0.197
Canny	26	0.468	0.273	0.300	0.146
laplacian	26	0.485	0.242	0.258	0.123
Feature Fusion	30	0.831	0.671	0.802	0.484

앞에서 설명한 바와 같이, 본 연구에서는 포트홀 윤곽과 질감을 보다 선명하게 표현하기 위해 CLAHE 기반 전처리를 적용하였다. CLAHE 전처리가 탐지 성능에 미치는 영향을 확인하기 위해, 기본 전처리만 사용한 모델과 CLAHE를 적용한 모델의 학습·검증 결과를 비교하였으며, 그 결과는 Figure 3과 Figure 4에 제시하였다.

Figure 3의 기본 모델에서는 epoch가 증가함에 따라 box loss와 class loss가 전반적으로 감소하고, precision, recall, mAP50 및 mAP50-95 지표가 전반적으로 향상되는 양상을 보인다. Figure 3과 Figure 4에서 두 모델 모두 일부 초기 epoch 구간에서 검증 class loss가 일시적으로 증가했다가 감소하는 구간이 관찰되지만, epoch가 진행될수록 손실은 감소하고 성능 지표는 향상되는 방향으로 수렴하는 경향을 나타낸다.

Figure 4의 CLAHE 전처리 모델의 경우, 학습 후반부에서 precision과 mAP 계열 지표가 기본 모델보다 상대적으로 높은 수준에서 유지되는 양상을 보인다. 특히 mAP50-95 기준으로 기본 모델의 값은 0.46571, CLAHE 전처리 모델은 0.52323으로, 절대값 기준 약 0.058, 상대적으로 약 12%의 증가가 나타난다. 반면 recall은 두 모델 모두 0.6~0.7 범위에서 유사한

수준을 유지하여, 본 실험에서는 주로 정밀도와 mAP 지표에서 차이가 두드러지는 것으로 확인되었다. 한편 epoch이 후반부로 진행될수록 train loss는 지속적으로 감소하는 반면, val loss는 큰 폭의 감소 없이 일정 범위에서 변동을 보이고 mAP 계열 지표 역시 특정 epoch 이후에는 뚜렷한 향상 없이 이전보다 다소 낮아지는 구간도 관찰된다. 이는 데이터셋 규모와 분포를 고려할 때 모델이 학습 데이터에 부분적으로 과적합되고 있을 가능성을 보여준다.

Figure 3 _ 기본 모델

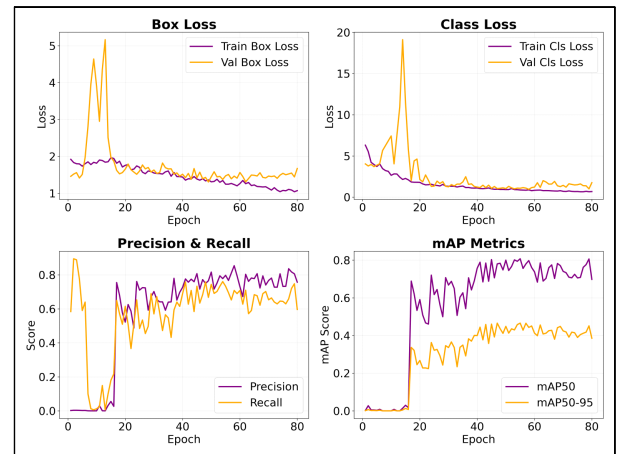


Figure 4 _ CLAHE 전처리

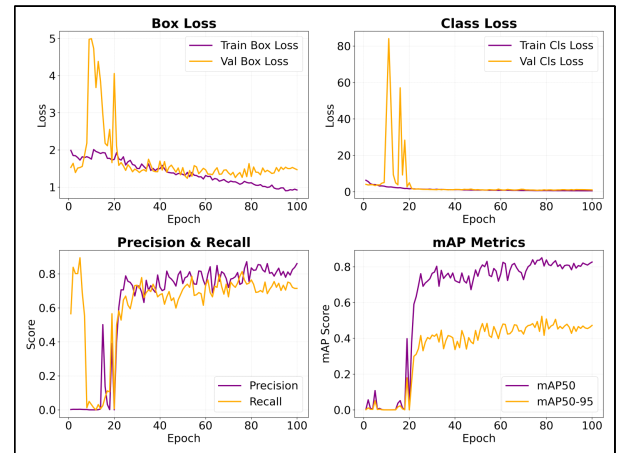


Table 2 모델 A과 B 성능

모델	mAP 50-95	mAP50	Precision	Recall
A	0.46	0.80	0.79	0.73
B	0.52	0.85	0.87	0.73

※ A: CLAHE 모델, B: CLAHE+Alb 모델

2.2.2 데이터 증강 및 정규화 방법

앞 절에서 관찰된 과적합 경향을 어느 정도 완화하기 위해 YOLOv8의 데이터 증강 하이퍼파라미터

진화를 통해 값을 얻었으며, 그 값들은 **Table 3**와 같다. HSV 색공간에서의 색상(hsv_h), 채도(hsv_s), 명도(hsv_v) 변환은 서로 다른 조명·색감을 모사하기 위한 것이며, translate와 scale은 포트홀이 영상 내에서 위치와 크기가 달라지는 상황을 반영하도록 설정하였다. 좌우 반전(fliplr)은 실제 주행 환경에서의 차선 방향 변화를 고려해 일정 확률로 적용되었고, Mosaic과 MixUp은 다양한 배경과 포트홀 조합을 만들어 데이터 다양성을 높이기 위해 상대적으로 높은 비율로 사용되었다. 반대로 CutMix, copy_paste 등은 실제 도로 환경과 괴리가 클 수 있어서 사용되지 않았다.

추가로, 원본 데이터 수를 보완하기 위해 일부 이미지는 Albumentations 라이브러리를 이용해 학습 전에 증강 이미지를 미리 생성한 뒤 기존 학습용 데이터셋에 함께 포함시켰다. 이를 통해 다양한 조건의 포트홀 사례를 인위적으로 늘려, 학습에 사용되는 데이터의 폭과 다양성을 넓히기 위한 추가적인 데이터 증강 과정으로 활용하였다.

Table 3 데이터 증강 하이퍼파라미터 설정 값

증강 기법	값
hsv_h	0.01358
hsv_s	0.43907
hsv_v	0.43997
degrees	0.0
translate	0.19653
scale	0.66799
shear	0.0
perspective	0.0
flipud	0.0
fliplr	0.4658
bgr	0.0
mosaic	0.82347
mixup	0.0677
cutmix	0.0
copy_paste	0.0
close_mosaic	10

Table 3에 정리한 데이터 증강 기법을 CLAHE 기반 모델에 적용하여 **Figure 4**와 동일한 학습 조건으로 다시 학습을 진행하였고, 그 결과를 **Figure 5**에 나타냈다. **Figure 5**를 **Figure 4**와 비교하면, mAP50-95 지표 기준으로 데이터 증강을 적용한 모델은 epoch 53에서 0.5117에 도달하고, CLAHE만 적용한 모델은 epoch 78에서 0.52323에 도달하여 데이터 증강을 적용한 경우 더 이른 시점에 성능이 상승하는 경향을 보인다. 또한 학습 후반부 곡선을 보면 val loss와 mAP 지표가 **Figure 4**에 비해 비교적 완만하게 변화하고, 일부 구간에서는 recall 값이 조금 더

높게 나타나 과적합이 이전보다 약해진 것으로 해석할 수 있다. 그럼에도 최고 mAP50-95 값만 놓고 보면 데이터 증강을 적용한 모델(0.5117)이 CLAHE만 적용한 모델(0.52323)보다 낮아 mAP 성능은 향상되지 않았다. 정리하면, 이 설정에서의 데이터 증강은 학습 진행 양상과 과적합 양상을 일정 부분 개선하는 데에는 기여하였으나, 전체적인 탐지 성능을 끌어올리는 역할을 하지는 못한 것으로 보인다.

Figure 5 _ CLAHE + 데이터 증강

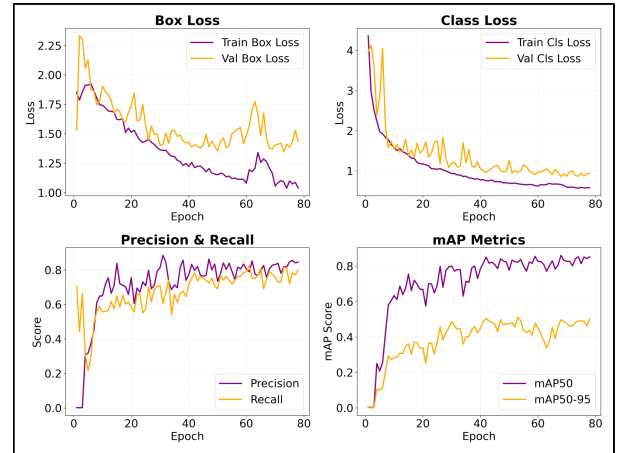


Table 4 모델 A 모델 B 성능

모델	mAP 50-95	mAP50	Precision	Recall
A	0.52	0.85	0.87	0.73
B	0.51	0.82	0.80	0.76

※ A: CLAHE+Alb 모델, B: CLAHE+Alb+증강 모델

2.3 모델 학습

2.3.1 최적화 기법 설정

앞 절에서 확인했듯이, CLAHE와 데이터 증강만으로는 mAP 성능 향상 폭이 제한적이고 과적합이 완전히 해소되지는 않았다. 이에 최적화 기법 측면에서의 개선 가능성을 확인하기 위해 YOLOv8의 기본 자동 선택 옵티마이저 대신 AdamW를 적용하였다. AdamW는 weight decay를 gradient 업데이트 과정과 분리하여 가중치에 직접 적용하는 방식으로 동작하며, 가중치가 불필요하게 커지는 현상을 줄여 과적합을 억제하는 데 도움이 되는 구조를 갖는다. AdamW의 가중치 업데이트 방식은 다음과 같이 정의된다.

$$\theta_{t+1} = \theta_t - \alpha \left(\frac{m_t}{\sqrt{v_t} + \epsilon} + \lambda \theta_t \right) \quad (1)$$

식(1)에서 θ_t 는 t번째 단계에서의 가중치이며, a 는 한 번의 학습 단계에서 가중치가 얼마나 크게 변할지를 결정하는 학습률이다. m_t 와 v_t 는 gradient 정보를 활용해 계산된 값으로, gradient는 모델이 손실을 줄이기 위해 가중치를 어느 방향으로 움직여야 하는지를 나타낸다. 식에 포함된 $\lambda\theta_t$ 부분은 가중치가 과도하게 커지는 것을 방지하기 위해, 학습 과정에서 일정 비율로 가중치 값을 줄이는 역할을 하며, λ 는 그 크기를 조절하는 계수이다. 이러한 구조를 통해 가중치가 지나치게 증가하는 문제를 억제하고 학습의 안정성을 유지하는 데 도움이 되어, 본 연구에서는 AdamW를 최적화 기법으로 적용하였다.

AdamW 옵티마이저를 적용할 때 사용한 주요 하이퍼파라미터 값은 YOLOv8의 하이퍼파라미터 진화 기능을 통해 탐색되었으며, 최종적으로 Table 5과 같이 설정하였다. 초기 학습률 lr0는 0.00184로 설정하여 지나치게 큰 gradient로 인한 발산을 피하면서도 일정 수준의 학습 속도를 확보하고자 하였고, 학습이 진행될수록 학습률이 점진적으로 감소하도록 최종 학습률 비율 lrf는 0.01851로 두었다. momentum은 0.8885로 설정하여 최근 기울기 정보를 일정 부분 반영하도록 하였으며, weight_decay는 0.00038로 두어 가중치가 과도하게 커지는 경향을 억제하였다. 초기 학습 단계에서 손실 값이 급격하게 변하는 현상을 완화하기 위해 warmup_epochs는 약 2.85 epoch, warmup_momentum은 0.91233으로 설정하여, 일정 기간 동안은 비교적 완만한 속도로 가중치가 갱신되도록 하였다.

Table 5 최적화 기법 하이퍼파라미터 설정 값

최적화 기법	값
lr0	0.00184
lrf	0.01851
momentum	0.8885
weight_decay	0.00038
warmup_epochs	2.84817
warmup_momentum	0.91233

Figure 6은 앞에서 사용한 CLAHE 전처리와 데이터 증강 설정은 그대로 유지한 상태에서, 최적화 기법만 AdamW 옵티마이저로 변경하여 학습한 결과를 보여준다. Figure 5와 비교하면 AdamW를 사용한 모델은 epoch 51에서 mAP50-95가 0.55576에 도달하여, 동일한 데이터 증강을 사용한 이전 모델의 최고 값인 0.5117(epoch 53)보다 절대값 기준 약 0.044, 상대적으로 약 8% 높은 성능을 보인다. 같은 시점에서 mAP50 역시 0.87547로 Figure 5의 0.82206보

다 향상되었고, precision은 약 0.80대 초반에서 0.89 수준으로 증가한 반면 recall은 두 모델 모두 0.76 전후의 비슷한 수준을 유지한다. 검증 손실 측면에서도 AdamW를 적용한 모델은 val box loss와 val cls loss, val dfl loss가 전반적으로 더 낮은 값에서 수렴하여 train loss와 val loss 사이의 간격이 Figure 5에 비해 줄어드는 양상을 보인다.

정리하면, AdamW를 사용한 설정은 CLAHE + 데이터 증강 기반 모델의 mAP50 및 mAP50-95 지표를 개선하고 검증 손실을 낮추어 이전 설정에 비해 과적합 양상을 일정 부분 완화하는 효과를 보였으나, 여전히 train-validation 간 차이가 남아 있고 recall이 크게 개선되지 않아 일부 포트홀을 놓치는 문제와 과적합을 완전히 해소했다고 보기는 어렵다.

Figure 6 _ CLAHE + 데이터 증강 + AdamW

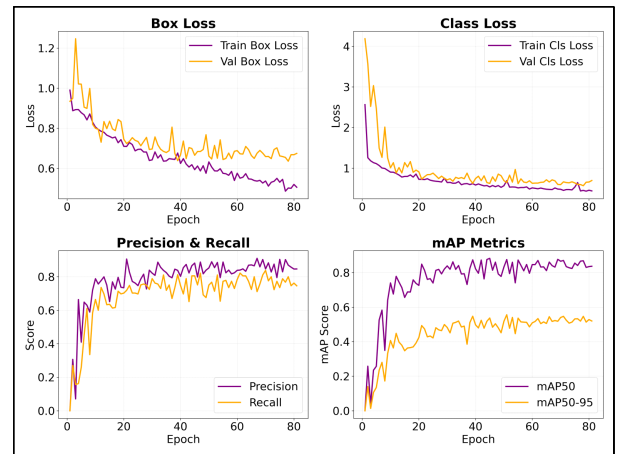


Table 6 모델 A과 모델 B의 성능 비교표

모델	mAP 50-95	mAP50	Precision	Recall
A	0.51	0.82	0.80	0.76
B	0.55	0.87	0.88	0.76

※ A: CLAHE+Alb, B: CLAHE+Alb+AdamW 모델

2.3.2 손실 함수 수정

본 연구에서는 box loss를 개선하기 위해 YOLOv8의 기본 box loss를 EIoU 기반 손실로 교체하고, 여기에 Focal weight를 곱한 Focal EIoU 방식을 사용하였다. EIoU는 단순 IoU 값을 사용하는 대신 두 박스의 중심 거리와 너비·높이 차이를 함께 고려하여 위치 오차와 크기 오차를 직접적으로 줄이도록 설계된 손실이다. Focal weight는 EIoU로부터 계산된 손실 값에 IoU의 거듭제곱 형태의 weight를 곱해, IoU 크기에 따라 각 예측이 최종 손실 계산에 얼마

나 반영되는지를 조절하는 역할을 한다. 다시 말해, EIoU는 박스의 기하학적 관계를 반영하는 box loss의 기본 형태이고, Focal weight는 예측 난이도에 따라 그 손실 크기를 제조정하는 가중 요소이다. 먼저 예측 박스 B_p 와 정답 박스 B_g 에 대해 IoU는 다음과 같이 정의된다.

$$IoU = \frac{B_p \cap B_g}{B_p \cup B_g} \quad (2)$$

식(3)에서 B_p 와 B_g 두 박스를 모두 포함하는 가장 작은 사각형을 C라고 하고, 가로와 세로 길이를 각각 c_w , c_h 라고 한다. 또한 두 박스 중심 사이의 거리 제곱을 d^2 , 예측 박스의 너비와 높이를 w_p , h_p , 정답 박스의 너비와 높이를 w_g , h_g 로 표기하면, EIoU는 다음과 같이 정의된다.

$$EIoU = IoU - \left(\frac{d^2}{c_w^2 + c_h^2} + \frac{(w_p - w_g)^2}{c_w^2} + \frac{(h_p - h_g)^2}{c_h^2} \right) \quad (3)$$

EIoU는 두 박스의 겹치는 정도를 나타내는 IoU와 두 박스 중심 간 거리, 너비·높이 차이를 함께 사용하도록 정의되며, 두 박스의 위치나 크기 차이가 클수록 더 작은 값을 갖는다. 수치 계산 과정에서는 c_w , c_h 가 매우 작아져 분모가 0 또는 0에 가까운 값이 되는 상황을 피하기 위해, 분모에 작은 상수 ϵ 를 더하고 c_w 와 c_h 가 0이 되지 않도록 최소값을 두어 연산을 안정화하였다. 실제 학습 과정에서는 EIoU 값이 이상치로 치우치지 않도록 $[-1, 1]$ 범위로 한번 더 제한(clamp)하였다.

본 연구에서 사용한 Focal EIoU 손실은 먼저 EIoU에 대한 손실을 다음과 같이 정의된다.

$$L_{EIoU} = 1 - EIoU \quad (4)$$

식(4)의 L_{EIoU} 에 IoU^γ 가중치를 곱하여 Focal EIoU 손실을 구성하였다. Focal EIoU 손실은 다음과 같이 정의 된다.

$$L_{FEIoU} = IoU^\gamma \times (1 - EIoU) \quad (5)$$

Focal EIoU 손실식에 사용되는 지수 γ 는 Focal weight의 세기를 조절하는 하이퍼파라미터이다.

$\gamma = 0$ 인 경우에는 $IoU^\gamma = 1$ 이 되어 Focal EIoU 손실이 단순 EIoU 손실과 동일해지며, 이때는 Focal 효과가 적용되지 않는다. γ 가 너무 작으면 IoU^γ 값이 1에 가까워져, 예측이 잘 맞은 경우와 많이 어긋난 경우가 손실에 거의 비슷한 비중으로 반영된다. 이때는 학습 초기 단계에서 손실 값의 변동 폭이 크고, 손실이 안정적으로 줄어들기까지 더 많은 예폭이 필요한 경향이 있었다. 반대로 γ 가 너무 크면 위치가 크게 어긋난 예측에 대해서는 손실 기여도가 빠르게 줄어들고, 이미 어느 정도 위치가 맞게 잡힌 예측에만 학습이 집중되는 양상이 나타났다. 그 결과, 초기에 잘못 잡힌 박스들의 위치가 충분히 보정되지 못해 Recall과 mAP50-95 개선 폭이 제한되는 문제가 관찰되었다.

본 연구에서는 γ 값을 여러 값을 설정하여 학습을 수행한 결과, 값이 지나치게 작을 때는 손실 곡선이 불안정하게 진동하고 수렴이 느려지는 문제가 있었고, 값이 지나치게 클 때는 잘못된 박스를 고쳐 나가는 능력이 떨어지는 문제가 있었다. 중간 수준의 값들 가운데 $\gamma = 0.5$ 에서 손실 곡선이 가장 안정적으로 감소하고, 검증 단계에서 box loss와 mAP 지표가 균형 있게 개선되는 것을 확인하여, $\gamma = 0.5$ 를 Focal EIoU 손실의 최종 하이퍼파라미터로 사용하였다.

손실 함수는 box loss, cls loss, dfl loss 세 부분에 가중치를 곱해 더한 형태로 사용하였다. 이때 사용한 가중치 값은 YOLOv8의 하이퍼파라미터 진화를 통해 탐색된 값으로, 최종적으로 box loss 3.8536, cls loss 0.39356, dfl loss 0.9893으로 설정하였으며 해당 값은 Table 7에 정리하였다. 기본 YOLOv8 설정(box 7.5, cls 0.5, dfl 1.5)과 비교하면 세 항목의 가중치를 전반적으로 낮추었다. cls loss는 포트홀/비포트홀 분류가 전체 손실을 지나치게 좌우하지 않도록 상대적으로 낮게 두었고, dfl loss는 위치 정보를 보완하는 역할을 고려해 box loss와 어느 정도 균형을 이루는 중간 정도의 값으로 설정하였다. 이러한 가중치 구성은 이후의 EIoU와 Focal EIoU 실험에도 동일하게 사용하였다.

Table 7 손실 함수 항목별 가중치 설정값

손실 항목	값
box	3.8536
cls	0.39356
dfl	0.9893

앞에서 설명한 손실 함수 설정을 적용하여 학습한 결과는 Figure 7과 Figure 8에 정리하였다. 두 실험

모두 CLAHE 전처리와 데이터 증강, AdamW 옵티마이저는 동일하게 두고, box loss만 달리하였다. Figure 7은 EIoU loss를 사용한 경우이고, Figure 8은 EIoU에 Focal 계수를 결합한 Focal EIoU loss를 사용한 경우이다. 이 두 그림을 먼저 비교함으로써, Focal 계수 도입이 EIoU 단독 대비 성능과 과적합 양상에 어떤 영향을 미치는지 확인하였다.

Figure 7과 Figure 8을 비교하면, mAP50-95의 최고 값은 EIoU에서 0.55859(epoch 98), Focal EIoU에서 0.56852(epoch 84)로 나타나 Focal EIoU가 약 0.01 높은 값을 보인다. mAP50 역시 EIoU의 0.87596에 비해 Focal EIoU는 0.88836으로, 전반적인 평균 정밀도가 소폭 개선된 양상을 보인다. precision과 recall 값을 함께 보면, EIoU는 recall이 0.85946으로 가장 높지만 precision이 0.84910까지 떨어져 포트홀을 놓치는 비율은 줄어들지만, 오검출이 늘어나면서 두 지표 사이의 균형이 한쪽으로 치우치는 모습을 보인다. 반면 Focal EIoU에서는 precision이 0.88787까지 회복되고 recall은 0.81622 수준으로 형성되어, EIoU보다는 낮지만 AdamW 기본 설정보다 높은 수준을 유지한다. 결과적으로 정밀도와 재현율의 균형이 EIoU보다 완만한 지점에서 수렴하는 것으로 해석할 수 있다. 검증 손실 곡선을 보면, Focal EIoU를 사용한 모델에서 val cls loss가 EIoU 모델보다 더 낮은 값 근처에서 안정적으로 유지되고, box loss 역시 비슷하거나 조금 낮은 수준을 보여준다. 이는 EIoU만 사용했을 때에 비해 오검출로 치우치는 경향이 다소 줄어들고, 분류 측면의 수렴이 보다 안정적으로 이루어진 것으로 해석할 수 있다.

이 가운데 상대적으로 성능과 지표 균형이 우수한 Focal EIoU를 기준으로 앞 절의 AdamW 기반 모델(Figure 6)과 비교하면 다음과 같다. mAP50-95 최고 값은 AdamW 모델에서 0.55576(epoch 51), Focal EIoU에서 0.56852(epoch 84)로, 절대값 기준 약 0.013 향상되었다. mAP50도 0.87547에서 0.88836으로 증가하여 평균 정밀도가 개선되었고, recall은 0.76757에서 0.81622로 상승하여 이전 설정보다 더 많은 포트홀을 검출하는 방향으로 변화하였다. precision은 AdamW 모델이 약 0.889, Focal EIoU가 0.88787 수준으로 거의 동일하게 유지되어, 정밀도를 크게 떨어뜨리지 않으면서 재현율과 mAP 계열 지표를 함께 끌어올린 것으로 정리할 수 있다. 손실 곡선 측면에서는 Focal EIoU를 적용한 경우 val cls loss와 val box loss가 전반적으로 더 낮은 수준에서 수렴하고, 학습 후반부에 train loss와 val loss 사이 간격도 이전 설정에 비해 다소 좁혀져 과적합 정도가 어느 정도 누그러진 것으로 보인다. 다만 두 곡선 사이의 차이가 여전히

히 남아 있어, 손실 함수를 수정한 것만으로 과적합이 완전히 해소되었다고 보기는 어렵다.

정리하면, Focal EIoU 손실은 AdamW와 EIoU를 사용한 모델에 비해 mAP50 및 mAP50-95를 향상시키고, 정밀도와 재현율의 균형을 일정 수준 유지하면서 과적합을 어느 정도 완화하는 효과를 보였다. 다만 연산량과 추론 속도(FPS)는 이전 설정과 거의 동일하므로, 실제 주행 환경에서의 실시간 적용을 위해서는 이후 절에서 다루는 것처럼 모델 구조 측면에서의 경량화와 속도 개선을 추가로 검토할 필요가 있다.

Figure 7 _ CLAHE + 데이터 증강 + AdamW + EIoU

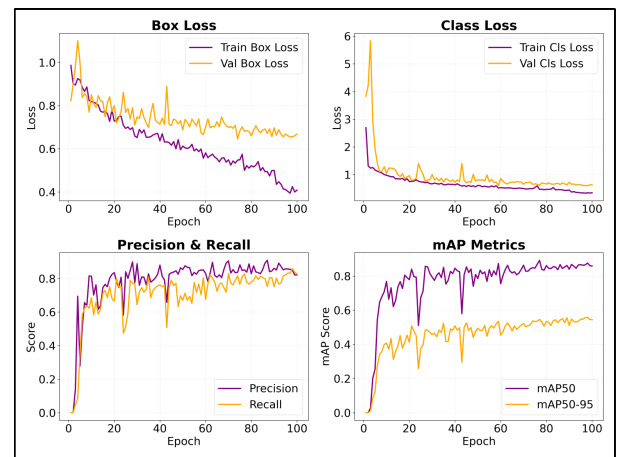


Figure 8 _ CLAHE + 데이터 증강 + AdamW + FEIoU

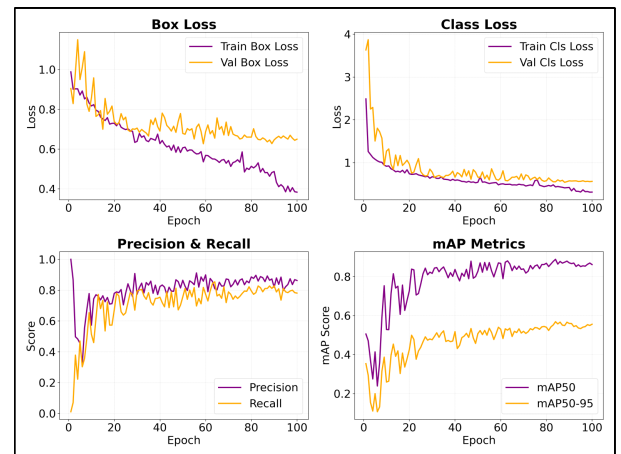


Table 8 모델 A, B, C 모델성능 비교표

모델	mAP 50-95	mAP50	Precision	Recall
A	0.55	0.87	0.88	0.76
B	0.55	0.87	0.84	0.85
C	0.56	0.88	0.88	0.81

※ A: CLAHE + Alb + AdamW

B: CLAHE + Alb + AdamW + EIoU

C: CLAHE + Alb + AdamW + Focal EI

2.3.3 백본 구조 수정

YOLOv8n 기본 모델은 Conv, C2f, SPPF, Upsample, Concat, Detect 모듈로 구성된 총 129개의 레이어로 이루어져 있으며, 백본은 P1(1/2)부터 P5(1/32)까지 다섯 단계의 해상도 변환을 거치는 구조를 갖는다. 이 과정에서 채널 수는 64→128→256→512→1024로 점차 확장되고, 특히 최종 단계인 P5(1/32)는 1024-channel Conv, 1024-channel C2f, SPPF로 구성되어 전체 모델에서 가장 높은 연산량을 요구하는 구간이다.

본 연구에서는 모델을 경량화하기 위해 백본의 최상위 단계인 P5 블록을 제거하였다. 구체적으로, 1024-channel Conv, 1024-channel C2f, SPPF로 이루어진 구간 전체를 삭제하고 채널 확장은 512채널에서 종료되도록 구조를 단순화하였다. 이에 따라 가장 해상도가 작은 특징 맵은 더 이상 생성되지 않으며, 이후 처리 과정에서도 해당 특징을 사용하던 연산이 모두 제거된다. 이러한 변경은 백본에서 연산량이 가장 큰 부분을 제거함으로써 전체 파라미터 수와 계산량을 크게 줄이는 효과가 있다.

헤드(head) 구조 또한 백본의 변경에 맞추어 재구성하였다. 원본 YOLOv8n 헤드는 P3(1/8), P4(1/16), P5(1/32)의 세 단계 특징 맵을 입력으로 사용하여 Upsample, Concat, C2f 연산을 단계적으로 수행하고, 마지막 Detect 모듈이 세 해상도를 모두 입력받도록 설계되어 있다. 그러나 백본에서 P5 단계가 제거되면서, 헤드에서 P5를 사용하던 Upsample, Concat, C2f, Conv 연산 역시 모두 사라지게 되었다.

수정된 헤드 구조는 백본의 P4 출력을 Upsample 하여 P3와 결합한 뒤 C2f 블록을 적용하는 방식으로 단순화되어 있으며, 이렇게 생성된 P3 특징 맵은 Conv(stride=2)를 이용해 Downsampling한 후 P4와 다시 결합하여 또 한 번 C2f 블록을 적용한다. 최종 Detect 모듈은 두 단계(P3, P4)만을 입력으로 사용하도록 축소되었으며, 결과적으로 원본의 P3,P4,P5 기반 3단계 검출 구조는 P3,P4 기반의 2단계 검출 구조로 변경되었다. 이러한 구조 변경은 헤드 전체의 연산 흐름을 단축시키고 연산량을 감소시켰다.

이와 같이 백본과 헤드를 단순화한 모델은 앞 절에서 분석한 Focal EIoU 손실 기반 모델과 동일한 전처리(CLAHE), 데이터 증강, AdamW, 손실 설정을 유지한 상태에서 학습하였으며, 그 결과를 **Figure 9**에 정리하였다. **Figure 9**를 보면 box loss와 cls loss는 epoch가 증가함에 따라 train과 val이 모두 감소하고, 학습 후반부에는 train loss는 약 0.4, val loss

는 약 0.6 수준에서 완만하게 수렴한다. 두 곡선의 간격이 완전히 사라지는 않지만, 감소 패턴 자체는 Focal EIoU 기준 모델과 크게 다르지 않아 학습 과정이 불안정하게 흔들리는 양상은 뚜렷하게 나타나지 않는다.

precision과 recall의 경우, 학습 후반부에 precision은 0.90에 가까운 수준까지 상승하고 recall은 약 0.81 근처에서 수렴하여, Focal EIoU 모델의 precision 0.88787, recall 0.81622와 비교했을 때 precision은 소폭 높아지고 recall은 비슷한 범위를 유지하는 것으로 나타났다. mAP 지표에서는 mAP50 최고 값이 0.88836에서 0.88030으로, mAP50-95는 0.56852에서 0.54652로 각각 약 0.008, 0.022 정도 낮아져 전체 성능 측면에서 일정 수준의 저하가 나타났다. 절대값 기준으로 큰 차이는 아니지만, mAP 계열 지표는 Focal EIoU 모델보다 다소 낮은 수준에서 수렴하는 반면 precision은 약간 더 높고 손실 곡선도 비교적 안정적으로 감소하는 양상을 보인다. 백본을 단순화한 이후 성능 저하는 비교적 작은 폭에 그치면서, 연산량을 줄이는 구조적 이점은 그대로 유지되었다. 한편 train과 val 손실 사이의 차이가 여전히 남아 있어, 구조를 단순화하더라도 과적합이 완전히 사라진 것은 아닌 것으로 보인다.

Figure 9 _ CLAHE + 데이터 증강 + AdamW + FIoU + 백본

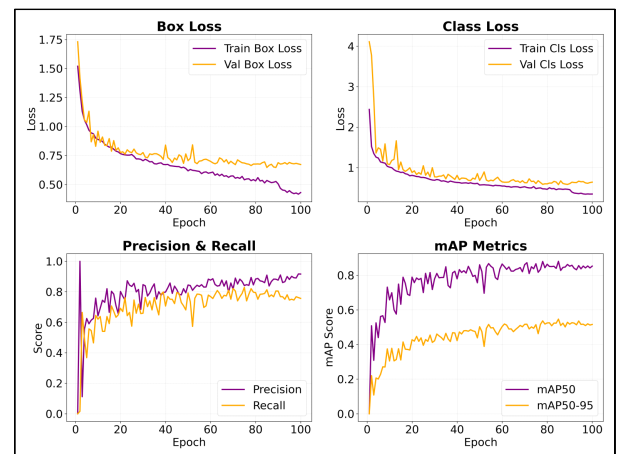


Table 9 모델 A, B 모델 성능 비교표

모델	mAP 50-95	mAP50	Precision	Recall	FPS (CPU)
A	0.56	0.88	0.88	0.81	9.68
B	0.54	0.88	0.90	0.81	13.84

※ A: CLAHE + Alb + AdamW + Focal EIoU

B: CLAHE + Alb + AdamW + Focal EIoU+백본

2.3.4 활성화 함수 수정

YOLOv8 기본 구조에서는 대부분의 convolution 블록에 활성화 함수로 SiLU를 사용한다. SiLU는 입력 x 에 대해 출력이 부드럽게 변하는 비선형 함수를 제공하여, 다층 신경망에서 작은 입력 변화에도 출력이 부드럽게 이어지도록 하여 다양한 패턴을 표현할 수 있도록 돕는다. 다만 sigmoid 연산이 포함되어 있어, 같은 구조에서 ReLU에 비해 처리해야 할 연산 수가 많고, 제한된 연산 자원을 사용하는 환경에서는 추론 속도에 불리하게 작용할 수 있다.

본 연구에서는 모델을 경량화하고 실시간 추론 속도를 고려하기 위해, YOLOv8의 기본 SiLU 활성화 함수를 ReLU로 교체하였다. ReLU는 0 이하의 입력은 모두 0으로 만들고, 0보다 큰 입력은 그대로 통과시키는 단순한 형태의 함수이다. 이 특성 덕분에 곱셈이나 지수 계산이 필요하지 않아 연산이 가볍고, 출력 값이 0이 되는 뉴런이 많아져 불필요한 활성화를 줄이는 효과가 있다. 이러한 이유로 ReLU는 경량 모델이나 실시간 처리가 필요한 상황에서 널리 사용되며, 본 연구에서도 동일한 방향에서 활성화 함수를 교체하였다. ReLU는 다음과 같이 정의된다.

$$ReLU(x) = \max(0, x) \quad (6)$$

앞 절에서 백본과 헤드를 단순화한 모델을 기준으로, 동일한 구조에서 활성화 함수를 변경했을 때 성능과 연산 효율이 어떻게 달라지는지 추가로 확인하였다. 활성화 함수 후보로는 기존 YOLOv8에서 사용하는 SiLU 대신, 연산이 단순한 ReLU 계열을 적용하였다. 이때 LeakyReLU(0.1)는 입력이 음수일 때에도 출력이 완전히 0으로 떨어지지 않도록 작은 기울기를 남겨 두는 ReLU의 변형으로, 음수 구간에서 정보가 완전히 사라지는 상황을 줄이면서도 연산 복잡도는 ReLU와 거의 동일하다는 점을 고려하여 함께 실험에 포함하였다. 전처리(CLAHE), 데이터 증강, AdamW, Focal EIoU 손실 및 box, cls, dfl 가중치는 백본 단순화 모델과 동일하게 유지하고, 활성화 함수만 LeakyReLU(0.1)와 ReLU로 각각 교체하여 학습하였으며, 그 결과를 Figure 10(LeakyReLU), Figure 11(ReLU)에 제시하였다.

Figure 10과 Figure 11을 비교하면, 두 설정 모두에서 box loss와 cls loss는 epoch가 증가함에 따라 train과 val이 전반적으로 감소하며 학습 후반부에 완만하게 수렴하는 경향을 보인다. loss 곡선의 형태는 유사하지만, mAP 지표에서는 차이가 나타난다. mAP50-95 기준 최고 값은 LeakyReLU(0.1) 모델이

0.51079, ReLU 모델이 0.52492로 ReLU 사용 시 약 0.014 높은 값을 기록하였으며, mAP50 역시 0.85551(LeakyReLU)와 0.87559(ReLU)로 ReLU 쪽이 다소 우세하였다. precision은 LeakyReLU(0.1) 모델이 0.89199로 ReLU(0.88404)보다 약간 높았지만, recall은 두 모델 모두 약 0.79 수준에서 수렴하여 큰 차이를 보이지 않았다. 전체적으로 볼 때 LeakyReLU(0.1)와 ReLU는 손실 곡선과 지표 추세가 비슷한 범위에 있으나, mAP50 및 mAP50-95 측면에서 ReLU가 약간 더 높은 성능을 보여, 백본 단순화 모델의 활성화 함수로는 ReLU를 채택하였다.

선택된 ReLU 기반 백본 단순화 모델을 앞 절의 SiLU 기반 백본 단순화 모델(Figure 9)과 비교하면, mAP50-95 기준 최고 값은 0.54652(SiLU)에서 0.52492(ReLU)로 약 0.022 감소하였고, mAP50 역시 0.88030에서 0.87559로 소폭 낮아졌다. precision과 recall도 각각 0.90937→0.88404, 0.81361→0.79459로 두 지표 모두 약간 감소하여, 정확도 측면에서는 SiLU 기반 모델이 여전히 우세한 것으로 나타났다. 반면 동일한 학습 환경에서 epoch당 평균 학습 시간은 ReLU를 사용할 때가 SiLU 대비 다소 짧게 관찰되어, 활성화 함수 연산이 단순해진 만큼 연산 효율은 일정 부분 개선된 것으로 볼 수 있다. 손실 곡선에서는 여전히 train과 val 사이에 차이가 남아 있어 과적합 경향이 완전히 사라졌다고 보기는 어렵고, 활성화 함수 변경만으로 이러한 경향을 근본적으로 해결하기에는 한계가 있음을 보여준다. 요약하면, ReLU는 SiLU 대비 약간의 성능 저하와 맞바꾸는 대신, 백본이 단순화된 모델에서 연산 속도를 조금 더 높이기 위한 선택으로 사용되었다.

Figure 10 _ CLAHE + 데이터 증강 + AdamW + FIoU + 백본+ Leaky ReLU

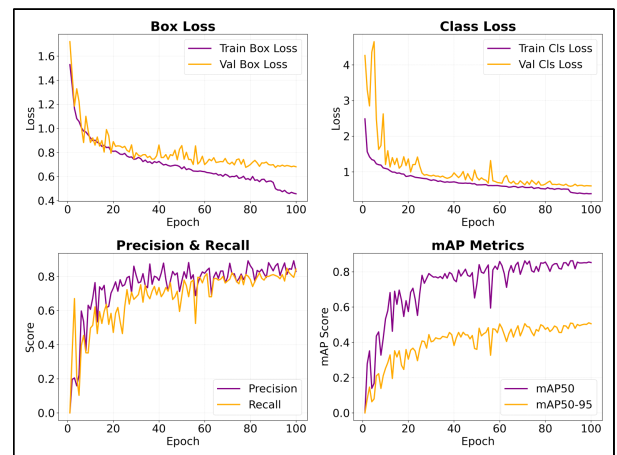


Figure 11 _ CLAHE + 데이터 증강 + AdamW + FElIoU + 백본 + ReLU

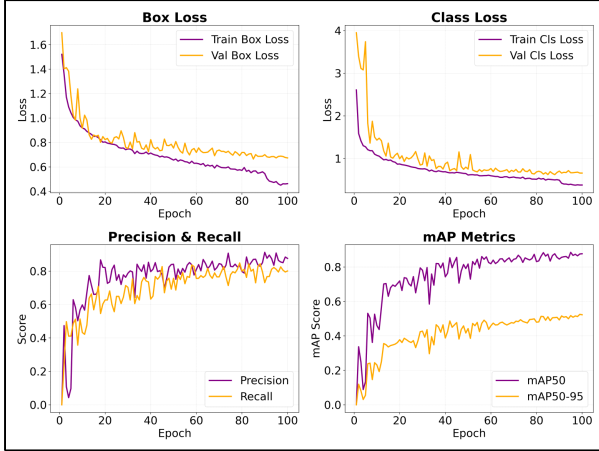


Table 10 모델 A, B, C모델 성능 비교표

모델	mAP 50-95	mAP50	Precision	Recall	FPS (CPU)
A	0.54	0.88	0.90	0.81	13.84
B	0.51	0.85	0.89	0.79	15.11
C	0.52	0.87	0.88	0.79	15.27

※ A: CLAHE + Alb + AdamW + FElIoU+백본

B: CLAHE + Alb + AdamW + FElIoU+백본+LeakyReLU

C: CLAHE + Alb + AdamW + FElIoU+백본+ReLU

III. 실험 결과

3.1 실험환경

본 프로젝트에서 수행한 모든 모델 학습과 평가는 동일한 하드웨어 환경에서 진행되었다. 사용한 시스템은 12세대 Intel(R) Core(TM) i5-12400 CPU와 NVIDIA GeForce RTX 3060 GPU가 장착된 데스크톱이며, 학습은 GPU를 사용하여 진행하였다. 실시간 추론 속도(FPS)는 동일 시스템에서 CPU만 사용하여 측정하였다.

3.2 학습 결과 분석

본 절에서는 앞 장에서 단계별로 제시한 전처리, 데이터 증강, 최적화 기법, 손실 함수, 백본·활성 함수 수정 결과를 종합하여 학습 경향을 정리한다. 전반적으로 모든 실험에서 epoch가 증가함에 따라 box loss와 cls loss는 감소하고, precision·recall·mAP50·mAP50-95 지표는 일정 수준까지 향상되는 공통된 수렴 양상을 보였다. 다만 대부분의 설정에서 학습 후반부에 train loss는 계속 낮아지는 반면 val loss와 mAP 계열 지표는 개선 폭이 제한적이거나 거의 변화하지 않아, 데이터셋 규모와 분포를 고려할 때

과적합 경향이 일정 수준 존재함을 확인할 수 있었다.

우선 기본 전처리만 사용한 모델과 비교했을 때, CLAHE 전처리를 적용한 모델(Figure 3, 4)은 포스트홀 윤곽과 질감을 강조함으로써 mAP50과 mAP50-95가 전반적으로 향상되었고, 특히 저대비 환경에서 p precision이 개선되는 경향을 보였다. 그러나 epoch가 후반부로 갈수록 train loss는 꾸준히 감소하는 데 비해 val loss와 mAP 계열 지표는 일정 범위에서 머무르거나 미세한 변동만 나타나, 전처리 개선만으로는 과적합 문제를 해결하기 어렵다는 한계를 드러냈다.

이후 데이터 증강과 AdamW, Focal EIoU 손실을 단계적으로 적용한 실험(Figure 5~8)에서는, 학습 초기 수렴 속도가 빨라지고 최고 mAP50-95 값이 기본 설정 대비 개선되는 양상이 관찰되었다. 특히 Focal EIoU 손실을 적용한 경우, EIoU 단독 대비 mAP50과 mAP50-95가 더 높은 값에서 수렴하고, precision과 recall의 균형도 보다 완만한 지점에 형성되어 손실 함수 관점에서의 개선 효과가 뚜렷하게 나타났다. 그럼에도 train과 val 손실 곡선 사이의 간격이 완전히 사라지는는 않았으며, 데이터 증강과 손실 함수 조정이 과적합을 어느 정도 줄이는 데에는 기여하지만 이를 근본적으로 제거하지는 못한다는 점도 함께 확인되었다.

백본 상위 블록을 제거하여 구조를 단순화한 모델(Figure 9)은 Focal EIoU 기반 모델과 비교했을 때 mAP50-95가 소폭 감소하는 대신 precision이 약간 높아지고, 손실 곡선도 큰 불안정 없이 완만하게 감소하는 형태를 유지하였다. 이는 연산량이 큰 P5 단계를 제거함으로써 일부 정확도 저하를 감수하는 대신, 비교적 안정적인 학습 특성을 유지한 채 모델을 경량화한 결과로 해석할 수 있다. 다만 train과 val loss 사이의 차이가 여전히 남아 있어, 구조 단순화만으로 과적합이 해소되었다고 보기는 어렵다.

마지막으로, 백본을 단순화한 동일한 모델 구조에서 활성 함수를 SiLU 대신 LeakyReLU(0.1)와 ReLU로 변경한 실험(Figure 10, 11)에서는, 두 설정 모두 앞 절에서 분석한 SiLU 기반 백본 단순화 모델과 비슷한 손실·지표 추세를 보였으며 mAP50 및 mAP50-95는 ReLU가 LeakyReLU(0.1)보다 다소 높은 값을 기록하였다. 한편 SiLU 기반 모델과 비교하면 mAP50-95, precision, recall이 전반적으로 조금씩 낮아지는 대신 epoch당 학습 시간이 줄어들어, 활성 함수 연산을 단순화했을 때 속도가 다소 빨라지는 대신 정확도가 약간 떨어지는 경향이 있음을 확인할 수 있었다.

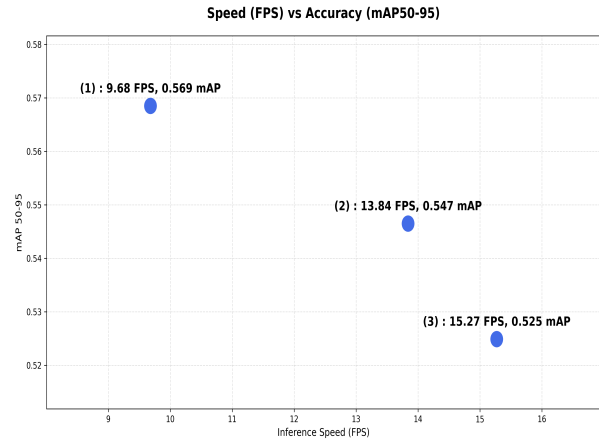
3.3 세 후보 모델 간 비교

앞 절에서 분석한 여러 실험 설정 가운데, 실용적인 관점에서 의미가 있다고 판단된 세 가지 모델을 선정하여 속도(FPS)와 정확도(mAP50-95)를 함께 비교하였다. 비교 대상은 (1) CLAHE + 데이터 증강 + AdamW + Focal EIoU 모델, (2) 첫 번째 구성에 백본 단순화를 적용한 모델, (3) 두 번째 구성에서 활성화 함수를 ReLU로 변경한 모델이다. 이 세 모델의 성능을 FPS - mAP50-95 산점도로 나타낸 것이 **Figure 12**이다.

Figure 12에서 가로축은 FPS를, 세로축은 mAP50-95를 나타낸다. 첫 번째 구성인 CLAHE + 데이터 증강 + AdamW + Focal EIoU 모델은 세 모델 중 mAP50-95가 가장 높지만 FPS가 가장 낮아, 정확도 측면에서 가장 유리한 성능을 보이는 모델이다. 두 번째 구성인 CLAHE + 데이터 증강 + AdamW + Focal EIoU + 백본 단순화 모델은 mAP50-95가 약간 낮아지는 대신 FPS가 눈에 띄게 증가하여, 일정 수준의 정확도를 유지하면서 추론 속도 측면에서 보다 유리한 성능을 보이는 모델이다. 세 번째 구성인 CLAHE + 데이터 증강 + AdamW + Focal EIoU + 백본 단순화 + ReLU 모델은 mAP50-95가 추가로 감소하지만 FPS가 가장 높은 위치에 놓여, 실시간 처리 측면에서 가장 유리한 성능을 보이는 모델이다.

이 세 모델을 종합적으로 비교하면, 첫 번째 모델은 포트홀 탐지 정확도를 가장 높게 확보할 수 있지만 FPS가 가장 낮아, 실시간 처리보다는 정확도를 우선하는 상황에 적합한 구성으로 볼 수 있다. 두 번째 모델은 mAP50-95가 다소 낮아지는 대신 속도가 눈에 띄게 빨라져, 일정 수준의 정확도를 유지하면서 추론 속도를 개선한 중간 단계의 선택지로 해석할 수 있다. 세 번째 모델은 세 구성 중 mAP50-95는 가장 낮지만 여전히 실용적인 수준의 탐지 성능을 유지하면서 가장 높은 FPS를 달성하였기 때문에, 본 연구에서 고려한 실시간 주행 환경을 생각했을 때 정확도와 속도 사이에서 가장 현실적인 균형점으로 판단하였다. 이에 따라 이후 3.4절에서는 CLAHE + 데이터 증강 + AdamW + Focal EIoU + 백본 단순화 + ReLU 모델을 최종 후보로 선정하고, 다음 절에서 원본 YOLOv8n 기본 모델과의 성능을 비교한다.

Figure 12 _ 모델 세 가지의 속도(FPS) - 정확도(mAP50-95) 비교



3.4 원본 YOLOv8n 모델과 비교

본 프로젝트에서 “원본 YOLOv8n 기본 모델”은 추가 전처리나 구조 변경 없이 YOLOv8n의 기본 백본 구조와 손실 함수, SiLU 활성화 함수를 사용하고 기본 전처리만 적용한 구성을 의미한다. 이 기본 모델은 동일한 데이터셋과 학습 조건에서 학습하였으며, 검증 세트 기준 성능은 mAP50-95 약 0.46, mAP50 약 0.80, precision 0.79, recall 0.73, FPS 9.68 수준으로 나타났다 (**Table 11**). 이때 저대비 구간이나 작은 포트홀에 대해서는 검출 성능이 충분하지 않고, train·val 손실 곡선 사이의 간격이 유지되는 등 과적합 경향도 일부 관찰되었다.

이에 비해 앞 절에서 최종 후보로 선정한 CLAHE + 데이터 증강 + AdamW + Focal EIoU + 백본 단순화 + ReLU 모델(이하 ‘최종 모델’)은 동일한 데이터셋을 사용하면서 전처리, 손실 함수, 백본 구조, 활성화 함수를 조합해 개선을 시도한 구성이다. **Table 1**에서 보듯이 최종 모델의 mAP50-95는 0.52, mAP50은 0.87, precision은 0.88, recall은 0.79, FPS는 15.27로, 원본 모델에 비해 탐지 성능과 추론 속도가 모두 향상되었다. 이는 원본 모델보다 오검출을 줄이면서 실제 포트홀을 더 많이 찾아내고, 초당 처리 가능한 프레임 수 역시 증가했음을 의미한다.

속도 측면에서는, 최종 모델이 백본 상위 블록 제거와 ReLU 활성화 함수 적용을 통해 연산량을 줄이도록 설계되었기 때문에, 동일 하드웨어 환경에서 원본 YOLOv8n 기본 모델보다 더 높은 FPS를 달성하였다. 한편 train과 val 손실 사이의 차이가 완전히 사라지는 않아, 전처리·손실 함수·구조 변경만으로 과적합 문제를 충분히 줄였다고 보기는 어렵다. 즉, 최종 모델은 원본 YOLOv8n 대비 포트홀 탐지 성능과 추론 속도 모두에서 개선을 보이지만, 향후 데이터 구성이나 모델 구조를 조정하는 추가 연구에 따

라 성능이 더 달라질 여지는 남아 있다.

Table 11 원본 모델과 최종 모델의 성능 비교

모델	mAP 50-95	mAP50	Precision	Recall	FPS (CPU)
원본	0.46	0.80	0.79	0.73	9.68
최종	0.52	0.87	0.88	0.79	15.27

IV. 결론

본 연구에서는 YOLOv8n을 기반으로 전처리, 데이터 증강, 최적화 기법, 손실 함수, 백본 구조, 활성화 함수를 단계적으로 변경해 가며 포트홀 탐지 성능 변화를 분석하였다. 특히 CLAHE 전처리와 Focal EIoU 손실, 백본 단순화, ReLU 활성화 함수를 조합하여 도로 표면에서 포트홀의 윤곽과 질감을 더 안정적으로 탐지하는 모델 구성을 찾고자 하였다.

실험 결과, CLAHE 전처리와 데이터 증강, Adam W, Focal EIoU를 적용한 뒤 백본을 단순화하고 ReLU를 사용한 모델은, 동일한 데이터셋과 조건에서 원본 YOLOv8n 기본 모델보다 mAP50-95, precision, recall이 모두 향상되고 데스크톱 CPU 환경에서 측정된 FPS 또한 증가하여, 포트홀 탐지 성능과 처리 속도 사이의 균형이 비교적 우수한 모델로 나타났다.

다만 이번 실험에 사용된 데이터의 양이 많지 않고, 학습 곡선에서 train과 val 손실의 차이가 끝까지 완전히 사라지지 않는 일정 수준의 과적합 경향이 남아 있는 것으로 보인다. 이러한 문제로 본 연구에서 얻은 성능이 다른 도로 환경이나 촬영 조건에서도 그대로 재현되는지는 추가적인 검증이 필요하다. 또한 비교 대상이 YOLOv8n 계열에 한정되어 있다는 점도 본 연구 결과를 해석할 때 고려해야 할 한계로 남는다.